

SIMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Course Code: CSA1203

Course Title: Computer Architecture

CO Assessed: CO2

Assessment: ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Weightage: 45% Total Marks: 25

CO2 Statement

Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)

CODE OF CONDUCT

I (G THARAKANTHREDDY / 192472032) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

ANNEXURE A

1. A processor performs arithmetic operations on signed 8-bit integers using 2's complement representation. Consider two numbers: +45 and -23. Analyze in detail how the system performs both addition and subtraction, including binary conversion, alignment of sign bits, and intermediate steps. Determine whether overflow occurs, and explain how the processor detects overflow and interprets the final result.

Answer:

The processor uses 8-bit 2's complement representation. The signed range is -128 to +127.

Binary conversion:

+45 = 00101101₂.

For -23, +23 = 00010111₂. Inverting the bits gives 11101000₂. Adding 1 gives 11101001₂. Therefore, -23 = 11101001₂.

Addition: +45 + (-23)

00101101

+ 11101001

1 00010110

The carry-out is discarded. The result is $00010110_2 = +22$.

Subtraction: $+45 - (-23)$

The 2's complement of -23 is $+23 = 00010111_2$.

00101101

+ 00010111

01000100

Therefore, $+45 - (-23) = +68$.

Overflow:

For addition, overflow occurs when two operands have the same sign and the result has the opposite sign. Here the operands have different signs, so overflow cannot occur. For subtraction, the operation becomes $+45 + 23 = +68$. Both operands and the result are positive and $+68$ is within the range -128 to $+127$. Hence, no overflow occurs.

2. A high-performance computing system replaces a ripple carry adder with a 4-bit carry look-ahead adder (CLA) to reduce computation delay. Given two binary inputs, analyze how generate and propagate signals are formed and how carries are computed in advance. Compare the time delay of CLA with ripple carry addition and justify, with reasoning, why CLA improves speed in modern processors.

Answer:

A 4-bit Carry Look-Ahead Adder reduces computation delay by calculating carry signals in advance.

For each bit i :

Generate: $G_i = A_i \cdot B_i$

Propagate: $P_i = A_i \oplus B_i$

Carry equations:

$$C_1 = G_0 + P_0C_0$$

$$C_2 = G_1 + P_1G_0 + P_1P_0C_0$$

$$C_3 = G_2 + P_2G_1 + P_2P_1G_0 + P_2P_1P_0C_0$$

$$C_4 = G_3 + P_3G_2 + P_3P_2G_1 + P_3P_2P_1G_0 + P_3P_2P_1P_0C_0$$

The sum is:

$$S_i = P_i \oplus C_i$$

In a ripple carry adder, each stage waits for the carry from the previous stage, so the delay increases as the number of bits increases. In a CLA, carries are calculated in parallel from the generate, propagate, and input carry signals. Therefore, the carry propagation delay is much smaller.

Thus, CLA improves processor speed because addition can be completed faster, making it useful in high-performance arithmetic units.

3. In a signed multiplication operation, a processor uses Booth's algorithm to multiply two numbers, -6 and $+5$. Analyze the algorithm step-by-step, including the role of the accumulator, multiplier, and Booth bit, along with arithmetic shifts. Explain how the algorithm efficiently handles signed numbers and reduces the number of addition/subtraction operations compared to traditional multiplication.

Answer:

Booth's algorithm performs signed multiplication directly using 2's complement representation.

$$-6 = 1010_2$$

$$+5 = 0101_2$$

Initial values:

$$A = 0000$$

$$Q = 0101$$

$$M = 1010$$

$$Q-1 = 0$$

Rules:

$$01 \rightarrow A = A + M$$

$$10 \rightarrow A = A - M$$

00 or 11 $\rightarrow$ No arithmetic operation

Then perform an arithmetic right shift.

Step 1:

$$Q_0Q_{-1} = 10.$$

$$A = 0000 - 1010 = 0110.$$

After arithmetic right shift:

$$A = 0011, Q = 0010, Q_{-1} = 1.$$

Step 2:

$$Q_0Q_{-1} = 01.$$

$$A = 0011 + 1010 = 1101.$$

After arithmetic right shift:

$$A = 1110, Q = 1001, Q_{-1} = 0.$$

Step 3:

$$Q_0Q_{-1} = 10.$$

$$A = 1110 - 1010 = 0100.$$

After arithmetic right shift:

$$A = 0010, Q = 0100, Q_{-1} = 1.$$

Step 4:

$$Q_0Q_{-1} = 01.$$

$$A = 0010 + 1010 = 1100.$$

After arithmetic right shift:

$$A = 1110, Q = 0010, Q_{-1} = 0.$$

Final product:

$$AQ = 11100010_2 = -30 \text{ in 8-bit 2's complement.}$$

$$\text{Therefore, } -6 \times +5 = -30.$$

Booth's algorithm handles signed numbers efficiently and reduces the number of additions/subtractions when the multiplier contains consecutive 1s.

4. A processor is required to perform division of two binary numbers (e.g., $13 \div 3$) using both restoring and non-restoring division algorithms. Analyze both methods step-by-step, showing intermediate remainders and quotient formation. Compare the two approaches in terms of number of steps, complexity, and efficiency, and explain why non-restoring division is often preferred in modern systems.

Answer:

$$\text{Dividend} = 13 = 1101_2$$

$$\text{Divisor} = 3 = 0011_2$$

Expected result:

$$13 \div 3 = 4 \text{ remainder } 1$$

$$\text{Quotient} = 0100_2$$

$$\text{Remainder} = 0001_2$$

Restoring division:

$$\text{Initial: } A = 00000, Q = 1101, M = 00011.$$

Step 1: Shift left $\rightarrow A = 00001$. Subtract M:

$$00001 - 00011 = 11110, \text{ which is negative. Restore } A \text{ to } 00001 \text{ and set quotient bit to } 0.$$

Step 2: Shift left $\rightarrow A = 00011$. Subtract M:

$$00011 - 00011 = 00000. \text{ Set quotient bit to } 1.$$

Step 3: Shift left $\rightarrow A = 00000$. Subtract M:

$$00000 - 00011 = 11101, \text{ which is negative. Restore } A \text{ and set quotient bit to } 0.$$

Step 4: Shift left $\rightarrow A = 00001$. Subtract M:

$$00001 - 00011 = 11110, \text{ which is negative. Restore } A \text{ and set quotient bit to } 0.$$

Final:

$$\text{Quotient} = 0100_2 = 4$$

$$\text{Remainder} = 0001_2 = 1$$

Non-restoring division:

The algorithm avoids immediately restoring a negative remainder.

Initial: $A = 00000$, $Q = 1101$, $M = 00011$.

Step 1: Shift left. A is non-negative, so subtract M :

$A = 00001 - 00011 = 11110$. Quotient bit = 0.

Step 2: Shift left. A is negative, so add M :

$A = 11100 + 00011 = 11111$. Quotient bit = 0.

Step 3: Shift left. A is negative, so add M :

$A = 11111 + 00011 = 00010$. Quotient bit = 1.

Step 4: Shift left. A is non-negative, so subtract M :

$A = 00100 - 00011 = 00001$. Quotient bit = 1.

Final:

Quotient = $0100_2 = 4$

Remainder = $0001_2 = 1$

Comparison:

Both methods use one main iteration per dividend bit. Restoring division may require an additional restoration operation whenever subtraction produces a negative remainder.

Non-restoring division avoids this immediate restoration and decides whether to add or subtract based on the sign of the partial remainder. Hence, it generally reduces unnecessary operations and is often preferred for efficient hardware implementation.

5. A scientific application requires precise handling of real numbers using the IEEE 754 standard for floating-point representation. Given a decimal number (e.g., -13.25), analyze how it is represented in both single precision and double precision formats, including sign, exponent, and mantissa fields. Further, explain how floating-point addition is performed between two such numbers, highlighting issues like normalization, rounding, and precision loss.

Answer:

Given decimal number = -13.25.

Binary conversion:

$13 = 1101_2$

$0.25 = 0.01_2$

Therefore, $13.25 = 1101.01_2$.

Normalized form:

$1101.01_2 = 1.10101_2 \times 2^3$.

Single precision (32-bit):

Sign bit = 1.

Problem (20%)	Understanding (20%)	Method Selection (20%)	Solution Process (25%)
Q1			
Q2			
Q3			
Q4			
Q5			

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately.	Understands problem with minor gaps.	Partial understanding; misses key elements.	Misinterprets or unable to understand problem.
Method Selection	20%	Selects most appropriate method/formula with strong justification.	Selects correct method with minor errors.	Inappropriate or partially correct method.	Unable to select method.
Solution Process	25%	Logical, systematic steps with correct approach throughout.	Mostly correct steps with minor mistakes.	Disorganized steps; multiple errors.	No clear process.
Accuracy of Calculation	20%	All calculations accurate; correct final answer.	Minor calculation errors.	Frequent errors; incorrect final answer.	No valid calculations.
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance.	Basic interpretation with minor omissions.	Limited or unclear interpretation.	No interpretation.